

Linux 장애 분석 및 트러블슈팅 보고서

OOM Crash

CPU Latency

Deadlock

실험 환경	Ubuntu 22.04 / aarch64 / UTM
서비스 계정	agent - admin (non - root)
대상 프로그램	agent - leak - app - arm64
서비스 포트	15034
최종 정상 설정	MEMORY_LIMIT=512MB CPU_MAX_OCCUPY=40% MULTI_THREAD_ENABLE=false

[Bug] OOM Crash - 메모리 증가 후 MemoryGuard에 의한 프로세스 종료

1. Description (현상 설명)

MEMORY_LIMIT=256MB 조건에서 MemoryWorker 실행 후 힙 사용량과 프로세스 RSS가 지속적으로 증가했다. 프로그램 내부 힙이 275MB에 도달하자 MemoryGuard가 제한 초과를 감지했고 프로세스가 SELF - TERMINATED 상태로 종료되었다.

2. Evidence & Logs (증거 자료)

외부 관측용 monitor - leak.sh 로그에서 동일 자식 프로세스의 RSS가 약 47.8MB에서 266.7MB까지 지속적으로 증가했다. 프로그램 로그에서는 Current Heap이 275MB에 도달한 직후 Memory limit exceeded 및 SELF - TERMINATED 메시지가 확인되었다.

관측 방법: pgrep -n -f로 대상 프로세스의 PID를 찾고, ps -p PID -o rss=로 RSS를 추출했다. ps가 출력한 RSS(KB)는 awk에서 1024로 나누어 MB로 변환한 뒤 timestamp와 함께 monitor - leak.log에 누적 기록했다.

```
agent-admin@kimseyoon:~$ tail -n 15 /var/log/agent-app/monitor-leak.log
[2026-09-19 10:40:46] PID:129166 CPU:0.5% MEM_RSS:0.9MB
[2026-09-19 10:40:49] PID:129166 CPU:0.4% MEM_RSS:0.9MB
[2026-09-19 10:40:52] PID:129166 CPU:0.3% MEM_RSS:0.9MB
[2026-09-19 10:40:55] PID:129166 CPU:0.3% MEM_RSS:0.9MB
[2026-09-19 10:40:58] PID:129166 CPU:0.3% MEM_RSS:0.9MB
[2026-09-19 10:53:18] PID:129482 CPU:0.8% MEM_RSS:47.8MB
[2026-09-19 10:53:21] PID:129482 CPU:0.6% MEM_RSS:79.8MB
[2026-09-19 10:53:24] PID:129482 CPU:0.5% MEM_RSS:108.4MB
[2026-09-19 10:53:27] PID:129482 CPU:0.4% MEM_RSS:141.7MB
[2026-09-19 10:53:30] PID:129482 CPU:0.4% MEM_RSS:166.7MB
[2026-09-19 10:53:33] PID:129482 CPU:0.4% MEM_RSS:191.7MB
[2026-09-19 10:53:36] PID:129482 CPU:0.3% MEM_RSS:216.7MB
[2026-09-19 10:53:39] PID:129482 CPU:0.3% MEM_RSS:241.7MB
[2026-09-19 10:53:42] PID:129482 CPU:0.3% MEM_RSS:266.7MB
```

그림 1. monitor - leak.sh 관측: 동일 PID의 MEM_RSS가 47.8MB - > 266.7MB로 지속 증가

```
2026-09-19 10:53:36,620 [INFO] [MemoryWorker] Current Heap: 200MB
2026-09-19 10:53:39,641 [INFO] [MemoryWorker] Current Heap: 225MB
2026-09-19 10:53:42,662 [INFO] [MemoryWorker] Current Heap: 250MB
2026-09-19 10:53:45,686 [INFO] [MemoryWorker] Current Heap: 275MB
2026-09-19 10:53:45,686 [CRITICAL] [MemoryGuard] Memory limit exceeded (275MB >= 256MB) / (Recommend Over 256MB)
2026-09-19 10:53:45,686 [CRITICAL] [MemoryGuard] Self-terminating process 129482 to prevent system instability.

>>> [SYSTEM] SELF-TERMINATED (Memory Limit Exceeded) <<<
Killed
```

그림 2. MEMORY_LIMIT=256MB: Current Heap 275MB에서 MemoryGuard가 제한 초과를 감지하고 프로세스 종료

3. Root Cause Analysis (원인 분석)

MemoryWorker가 메모리를 계속 확보하면서 프로세스의 실제 상주 메모리(RSS)가 증가했다. 설정된 256MB 한계보다 힙 사용량이 커지자 애플리케이션 내부 MemoryGuard가 보호 동작을 수행한 것이 직접적인 종료 원인이다. 이는 커널 OOM Killer가 동작했다는 증거와는 구분해야 하며, 본 실습 로그에서 확인되는 직접 원인은 애플리케이션의 MemoryGuard이다.

메모리 사용이 제한 없이 증가하면 시스템의 가용 메모리가 감소하여 같은 서버의 다른 프로세스에도 영향을 줄 수 있다. 따라서 MemoryGuard가 임계치를 초과한 프로세스를 종료하는 것은 메모리 고갈이 시스템 전체 장애로 확산되는 것을 막기 위한 보호 동작으로 볼 수 있다.

4. Workaround & Verification (조치 및 검증)

최종적으로 MEMORY_LIMIT을 512MB로 조정했다. 정상 설정(512MB / 40% / false)에서는 힙이 525MB에 도달했을 때 프로세스를 종료하지 않고 cleanup을 수행했으며, Memory Cache Flushed 및 MEMORY RECOVERED 이후 작업이 계속 진행되었다. 256MB 조건에서는 약 33초 후 종료되었지만 최종 정상 설정에서는 1분 이상 실행되며 메모리 회수 후에도 생존하는 것을 확인했다.

단, 이 After 결과는 MEMORY_LIMIT만 단독으로 변경한 통제 실험이 아니라 CPU_MAX_OCCUPY와 MULTI_THREAD_ENABLE까지 조정된 최종 정상 설정에서의 검증 결과이다.

```
2026-09-19 14:35:08,535 [INFO] [Thread-C] Resumed. Calculating... (100%)
2026-09-19 14:35:08,586 [INFO] [Scheduler] All tasks completed.
2026-09-19 14:35:08,591 [INFO] [MemoryWorker] Current Heap: 259MB
2026-09-19 14:35:08,591 [INFO] [CpuWorker] Started. Maximum CPU Limit: 40%
2026-09-19 14:35:08,592 [INFO] [CpuWorker] Current Load: 5.00%
2026-09-19 14:35:11,614 [INFO] [MemoryWorker] Current Heap: 50MB
2026-09-19 14:35:11,783 [INFO] [CpuWorker] Current Load: 14.65%
2026-09-19 14:35:14,635 [INFO] [MemoryWorker] Current Heap: 75MB
2026-09-19 14:35:14,822 [INFO] [CpuWorker] Current Load: 18.91%
2026-09-19 14:35:17,684 [INFO] [MemoryWorker] Current Heap: 100MB
2026-09-19 14:35:17,943 [INFO] [CpuWorker] Current Load: 19.21%
2026-09-19 14:35:20,714 [INFO] [MemoryWorker] Current Heap: 125MB
2026-09-19 14:35:21,060 [INFO] [CpuWorker] Current Load: 28.94%
2026-09-19 14:35:23,744 [INFO] [MemoryWorker] Current Heap: 150MB
2026-09-19 14:35:24,190 [INFO] [CpuWorker] Current Load: 27.68%
2026-09-19 14:35:26,777 [INFO] [MemoryWorker] Current Heap: 175MB
2026-09-19 14:35:27,314 [INFO] [CpuWorker] Current Load: 35.88%
2026-09-19 14:35:29,884 [INFO] [MemoryWorker] Current Heap: 200MB
2026-09-19 14:35:30,435 [INFO] [CpuWorker] Current Load: 37.84%
2026-09-19 14:35:32,550 [INFO] [CpuWorker] Peak reached (40.00%). Starting cooldown...
2026-09-19 14:35:32,835 [INFO] [MemoryWorker] Current Heap: 225MB
2026-09-19 14:35:33,556 [INFO] [CpuWorker] Current Load: 40.86%
2026-09-19 14:35:35,862 [INFO] [MemoryWorker] Current Heap: 250MB
2026-09-19 14:35:36,680 [INFO] [CpuWorker] Current Load: 33.35%
2026-09-19 14:35:38,891 [INFO] [MemoryWorker] Current Heap: 275MB
2026-09-19 14:35:39,880 [INFO] [CpuWorker] Current Load: 29.86%
2026-09-19 14:35:41,915 [INFO] [MemoryWorker] Current Heap: 300MB
2026-09-19 14:35:42,912 [INFO] [CpuWorker] Current Load: 27.79%
2026-09-19 14:35:44,943 [INFO] [MemoryWorker] Current Heap: 325MB
2026-09-19 14:35:46,037 [INFO] [CpuWorker] Current Load: 24.78%
2026-09-19 14:35:47,973 [INFO] [MemoryWorker] Current Heap: 350MB
2026-09-19 14:35:49,164 [INFO] [CpuWorker] Current Load: 22.57%
2026-09-19 14:35:51,081 [INFO] [MemoryWorker] Current Heap: 375MB
2026-09-19 14:35:52,290 [INFO] [CpuWorker] Current Load: 18.31%
2026-09-19 14:35:54,028 [INFO] [MemoryWorker] Current Heap: 400MB
2026-09-19 14:35:55,414 [INFO] [CpuWorker] Current Load: 12.79%
2026-09-19 14:35:57,040 [INFO] [MemoryWorker] Current Heap: 425MB
2026-09-19 14:35:57,531 [INFO] [CpuWorker] Cooldown complete (5.00%). Resuming load increase...
2026-09-19 14:35:58,541 [INFO] [CpuWorker] Current Load: 5.00%
2026-09-19 14:36:00,062 [INFO] [MemoryWorker] Current Heap: 450MB
2026-09-19 14:36:01,666 [INFO] [CpuWorker] Current Load: 6.99%
2026-09-19 14:36:03,092 [INFO] [MemoryWorker] Current Heap: 475MB
2026-09-19 14:36:04,790 [INFO] [CpuWorker] Current Load: 13.86%
2026-09-19 14:36:06,118 [INFO] [MemoryWorker] Current Heap: 500MB
2026-09-19 14:36:07,913 [INFO] [CpuWorker] Current Load: 16.17%
2026-09-19 14:36:09,144 [INFO] [MemoryWorker] Current Heap: 525MB
2026-09-19 14:36:09,144 [WARNING] [MemoryWorker] Memory Usage Reached Limit (525MB). Starting cleanup...
2026-09-19 14:36:09,160 [INFO] [System] Memory Cache Flushed. Process Stabilized.
>>> [SYSTEM] MEMORY RECOVERED (Cache Cleared) <<<
2026-09-19 14:36:11,030 [INFO] [CpuWorker] Current Load: 25.87%
2026-09-19 14:36:14,165 [INFO] [CpuWorker] Current Load: 34.27%
2026-09-19 14:36:14,198 [INFO] [MemoryWorker] Current Heap: 25MB
2026-09-19 14:36:16,282 [INFO] [CpuWorker] Peak reached (40.00%). Starting cooldown...
2026-09-19 14:36:17,225 [INFO] [MemoryWorker] Current Heap: 50MB
```

그림 3. 최종 정상 설정: 525MB 도달 후 cleanup 및 MEMORY RECOVERED, 이후 작업 지속

4

```

>>> Starting Agent Boot Sequence...
[1/6] Checking User Account [OK]
... Running as service user 'agent-admin' (uid=1001)
[2/6] Verifying Environment Variables [OK]
... All required Envs correct
[3/6] Checking Required Files [OK]
... Verified 'secret.key' with correct key string.
[4/6] Checking Port Availability [OK]
... Port 15034 is available.
[5/6] Verifying Log Permission [OK]
... Log directory is writable: /var/log/agent-app
[6/6] Verifying Mission Environment [OK]
... MEMORY_LIMIT=512MB, CPU_MAX_OCCUPY=80%, MULTI_THREAD_ENABLE=True
-----
All Boot Checks Passed!
Agent READY
2026-09-19 13:49:38,131 [INFO] [SafetyGuard] Process priority lowered (nice=10).
2026-09-19 13:49:38,131 [INFO] Agent listening at port 15034

=====
[ Agent Initiate ] Resource Check
=====
Terminal Limit: 512MB [ OK ]
Limit: 80% [ WARNING: Recommend Under 50% ]
[ INFO ] Concurrency: True [ WARNING ]
=====
>>> SYSTEM WARNING: POTENTIAL DEADLOCK IN CONCURRENT MODE.
=====
2026-09-19 13:49:40,135 [INFO] [CpuWorker] Started. Maximum CPU Limit: 80%
2026-09-19 13:49:40,135 [INFO] [CpuWorker] Current Load: 5.00%
2026-09-19 13:49:43,257 [INFO] [CpuWorker] Current Load: 5.52%
2026-09-19 13:49:46,376 [INFO] [CpuWorker] Current Load: 13.75%
2026-09-19 13:49:49,496 [INFO] [CpuWorker] Current Load: 21.47%
2026-09-19 13:49:52,621 [INFO] [CpuWorker] Current Load: 29.69%
2026-09-19 13:49:55,744 [INFO] [CpuWorker] Current Load: 32.76%
2026-09-19 13:49:58,869 [INFO] [CpuWorker] Current Load: 39.62%
2026-09-19 13:50:01,993 [INFO] [CpuWorker] Current Load: 47.54%
2026-09-19 13:50:05,109 [INFO] [CpuWorker] Current Load: 54.37%
2026-09-19 13:50:05,214 [CRITICAL] [CpuWorker] CPU Threshold Violated! (54.37%).
>>> [SYSTEM] WATCHDOG: INITIATING EMERGENCY ABORT (SIGTERM) <<<
Terminated

```

그림 5. CPU 장애 실행 로그: CpuWorker 부하 증가 및 Watchdog에 의한 종료 과정

3. Root Cause Analysis (원인 분석)

직접적인 종료 원인은 CpuWorker가 내부 부하 판정값을 임계 상태로 판단하여 Watchdog 보호 로직을 실행한 것이다. Linux top/ps 관측치는 1%대였으므로 내부 Current Load의 계산 방식은 OS의 %CPU와 다르다고 판단했다. 제공된 실행 파일의 내부 계산 방식을 역공학하지 않았으며, 관측된 두 지표의 차이를 그대로 기록했다.

일반적으로 단일 프로세스가 CPU를 장시간 과도하게 점유하면 OS scheduler가 CPU 시간을 분배하더라도 다른 프로세스의 실행 기회와 응답성이 저하될 수 있다. 따라서 Watchdog을 이용한 종료는 비정상적인 CPU 점유가 장시간 지속되어 시스템 응답성을 떨어뜨리는 상황을 제한하기 위한 보호 동작으로 볼 수 있다.

4. Workaround & Verification (조치 및 검증)

CPU_MAX_OCCUPY를 80%에서 40%로 변경했다. 최종 정상 설정에서는 CPU 항목이 [OK]로 판정되었고, CpuWorker가 40.00%에 도달했을 때 종료 대신 Starting cooldown으로 전환된 뒤 부하가 감소하고 다시 작업을 이어갔다. 따라서 80% 조건의 Watchdog SIGTERM과 달리 40% 조건에서는 CPU 관련 강제 종료가 발생하지 않았다.

이 After 역시 CPU_MAX_OCCUPY만 단독으로 변경한 통제 실험이 아니라 최종 정상 설정에서의 검증 결과이다.

```

agent-admin@insepyon:~/agent-app$ export MULTI_THREAD_ENABLE=false
./agent-task-app-arm64
>>> Starting Agent Boot Sequence...
[1/6] Checking User Account [OK]
... Running as service user 'agent-admin' (uid=1001)
[2/6] Verifying Environment Variables [OK]
... All required Envs correct
[3/6] Checking Required Files [OK]
... Verified 'secret.key' with correct key string.
[4/6] Checking Port Availability [OK]
... Port 15034 is available.
[5/6] Verifying Log Permission [OK]
... Log directory is writable: /var/log/agent-app
[6/6] Verifying Mission Environment [OK]
... MEMORY_LIMIT=512MB, CPU_MAX_OCCUPY=40%, MULTI_THREAD_ENABLE=False
-----
All Boot Checks Passed!
Agent READY
2026-09-19 14:35:05,482 [INFO] [SafetyGuard] Process priority lowered (nice=10).
2026-09-19 14:35:05,482 [INFO] Agent listening at port 15034
=====
[ Agent Initiate ] Resource Check
=====
[ MEMORY ] Limit: 512MB [ OK ]
[ CPU ] Limit: 40% [ OK ]
[ THREAD ] concurrency: False [ OK ]
=====
>>> SYSTEM STATUS: STABLE. STARTING WORKLOAD MONITORING...
=====
2026-09-19 14:35:07,493 [INFO] >>> Scenario Selected: [Healthy System Monitoring]
>>> [SYSTEM] ALL CONFIGURATIONS OPTIMAL. RUNNING STABILITY TEST... <<<
2026-09-19 14:35:07,493 [INFO] [Scheduler] Task Scheduler Initialized.
2026-09-19 14:35:07,493 [INFO] [Scheduler] Registered Tasks: ['Thread-A', 'Thread-B', 'Thread-C']
2026-09-19 14:35:07,493 [INFO] [Scheduler] Starting task execution...
2026-09-19 14:35:07,493 [INFO] [Thread-A] Task Started. Calculating... (20%)
2026-09-19 14:35:07,546 [INFO] [Thread-A] Calculating... (40%)
2026-09-19 14:35:07,597 [INFO] [Thread-A] Preempted. Progress saved at (40%)
2026-09-19 14:35:07,648 [INFO] [Thread-B] Task Started. Calculating... (20%)
2026-09-19 14:35:07,699 [INFO] [Thread-B] Calculating... (40%)
2026-09-19 14:35:07,753 [INFO] [Thread-B] Preempted. Progress saved at (40%)
2026-09-19 14:35:07,804 [INFO] [Thread-C] Task Started. Calculating... (20%)
2026-09-19 14:35:07,855 [INFO] [Thread-C] Calculating... (40%)
2026-09-19 14:35:07,906 [INFO] [Thread-C] Preempted. Progress saved at (40%)
2026-09-19 14:35:07,958 [INFO] [Thread-A] Resumed. Calculating... (60%)
2026-09-19 14:35:08,013 [INFO] [Thread-A] Calculating... (80%)
2026-09-19 14:35:08,065 [INFO] [Thread-A] Preempted. Progress saved at (80%)
2026-09-19 14:35:08,118 [INFO] [Thread-B] Resumed. Calculating... (60%)
2026-09-19 14:35:08,169 [INFO] [Thread-B] Calculating... (80%)
2026-09-19 14:35:08,221 [INFO] [Thread-B] Preempted. Progress saved at (80%)
2026-09-19 14:35:08,275 [INFO] [Thread-C] Resumed. Calculating... (60%)
2026-09-19 14:35:08,328 [INFO] [Thread-C] Calculating... (80%)
2026-09-19 14:35:08,380 [INFO] [Thread-C] Preempted. Progress saved at (80%)
2026-09-19 14:35:08,431 [INFO] [Thread-A] Resumed. Calculating... (100%)
2026-09-19 14:35:08,483 [INFO] [Thread-B] Resumed. Calculating... (100%)

```

그림 6. 최종 정상 설정(512MB / 40% / false): CPU 40% [OK], Healthy System Monitoring 및 정상 작업 진행

[Bug] Deadlock - 두 작업 스레드의 상호 Lock 대기

1. Description (현상 설명)

MEMORY_LIMIT=512MB, CPU_MAX_OCCUPY=40%, MULTI_THREAD_ENABLE=true 조건에서 동시 처리 스레드가 실행되었다. Worker - Thread - 1은 Shared_Memory_A를 획득한 뒤 Socket_Pool_B를 기다렸고, Worker - Thread - 2는 Socket_Pool_B를 획득한 뒤 Shared_Memory_A를 기다렸다. 두 스레드가 모두 BLOCKED 상태가 된 이후 로그 진행이 멈췄다.

2. Evidence & Logs (증거 자료)

프로그램 로그에 각 스레드의 LOCK ACQUIRED와 상대 자원에 대한 WAITING/BLOCKED가 연속으로 기록되었다. ps -ef로 부모/자식 프로세스가 종료되지 않고 존재함을 확인했고, ps -L로 자식 프로세스 내부에 메인 스레드와 두 작업 스레드가 존재하며 모두 %CPU 0.0 상태임을 확인했다.

진단 순서: (1) 로그가 WAITING/BLOCKED에서 더 이상 진행되지 않는 것을 확인하고, (2) ps -ef로 PID가 살아 있음을 확인한 뒤, (3) ps -L -p PID로 TID, CPU, MEM, STAT를 확인했다. (4) 프로세스는 살아 있지만 스레드가 진행하지 않는 상태와 A -> B, B -> A의 상호 Lock 대기 로그를 결합해 Deadlock으로 판단했다.

```
agent-admin@kimseeyoon:~/agent-app$ export CPU_MAX_OCCUPY=40
./agent-leak-app-arm64
>>> Starting Agent Boot Sequence...
[1/6] Checking User Account [OK]
... Running as service user 'agent-admin' (uid=1001)
[2/6] Verifying Environment Variables [OK]
... All required Envs correct
[3/6] Checking Required Files [OK]
... Verified 'secret.key' with correct key string.
[4/6] Checking Port Availability [OK]
... Port 15034 is available.
[5/6] Verifying Log Permission [OK]
... Log directory is writable: /var/log/agent-app
[6/6] Verifying Mission Environment [OK]
... MEMORY_LIMIT=512MB, CPU_MAX_OCCUPY=40%, MULTI_THREAD_ENABLE=True
-----
All Boot Checks Passed!
Agent READY
2026-09-19 14:28:27,444 [INFO] [SafetyGuard] Process priority lowered (nice=10).
2026-09-19 14:28:27,444 [INFO] Agent listening at port 15034

=====
[ Agent Initiate ] Resource Check
=====
[ MEMORY ] Limit: 512MB [ OK ]
[ CPU ] Limit: 40% [ OK ]
[ THREAD ] Concurrency: True [ WARNING ]
=====
>>> SYSTEM WARNING: POTENTIAL DEADLOCK IN CONCURRENT MODE.
=====

2026-09-19 14:28:29,455 [WARNING] [AgentWorker] Initializing concurrent transaction processors...
2026-09-19 14:28:29,482 [WARNING] [System] CAUTION: Strict resource locking is enabled.
2026-09-19 14:28:34,482 [INFO] [Worker-Thread-1] Process Started. Attempting to lock [Shared_Memory_A]...
2026-09-19 14:28:34,482 [INFO] [AgentWorker][Worker-Thread-1] LOCK ACQUIRED: [Shared_Memory_A]. (Holding...)
2026-09-19 14:28:34,482 [INFO] [AgentWorker][Worker-Thread-1] Processing critical data in Memory A...
2026-09-19 14:28:34,482 [INFO] [AgentWorker][Worker-Thread-2] Process Started. Attempting to lock [Socket_Pool_B]...
2026-09-19 14:28:34,482 [INFO] [AgentWorker][Worker-Thread-2] LOCK ACQUIRED: [Socket_Pool_B]. (Holding...)
2026-09-19 14:28:34,482 [INFO] [AgentWorker][Worker-Thread-2] Establishing network connections in Pool B...
2026-09-19 14:28:34,482 [INFO] [AgentWorker] Waiting for worker threads to complete transactions...
2026-09-19 14:28:36,493 [INFO] [AgentWorker][Worker-Thread-2] Need resource [Shared_Memory_A] to write logs.
2026-09-19 14:28:36,493 [INFO] [AgentWorker][Worker-Thread-2] WAITING for [Shared_Memory_A]... (Status: BLOCKED)
2026-09-19 14:28:36,493 [INFO] [AgentWorker][Worker-Thread-1] Need resource [Socket_Pool_B] to finish job.
2026-09-19 14:28:36,493 [INFO] [AgentWorker][Worker-Thread-1] WAITING for [Socket_Pool_B]... (Status: BLOCKED)
```

그림 7. Deadlock 발생: 각 스레드가 서로 다른 Lock을 보유한 채 상대 자원을 WAITING/BLOCKED

```
agent-admin@kimseeyoon:~$ ps -ef | grep '[a]gent-leak-app-arm64'
agent-a+ 131892 3908 0 14:28 pts/1 00:00:00 ./agent-leak-app-arm64
agent-a+ 131893 131892 0 14:28 pts/1 00:00:00 ./agent-leak-app-arm64
agent-admin@kimseeyoon:~$
```

그림 8. ps -ef: 장애 중에도 부모 PID 131892와 자식 PID 131893이 살아 있음

```

agent-admin@kimseyoon:~$ ps -L -p 131893 -o pid,tid,ppid,pcpu,pmem,rss,stat,comm
  PID      TID      PPID  %CPU  %MEM   RSS  STAT  COMMAND
 131893  131893   131892   0.0   0.4  16852  SNL+  agent-leak-app-
 131893  131895   131892   0.0   0.4  16852  SNL+  agent-leak-app-
 131893  131896   131892   0.0   0.4  16852  SNL+  agent-leak-app-
agent-admin@kimseyoon:~$

```

그림 9. ps -L: 자식 프로세스 내 3개 TID 확인, 각 스레드 %CPU 0.0으로 진행 정체

3. Root Cause Analysis (원인 분석)

Thread - 1은 자원 A를 보유한 상태에서 B를 기다리고, Thread - 2는 자원 B를 보유한 상태에서 A를 기다려 순환 대기가 형성되었다. 상호배제(Mutual Exclusion), 점유 대기(Hold and Wait), 비선점(No Preemption), 순환 대기(Circular Wait)의 네 가지 Coffman 조건이 모두 성립하는 전형적인 Deadlock 구조이다. 프로세스 자체는 살아 있지만 두 작업 스레드가 서로의 Lock 해제를 기다리므로 유의미한 작업 진행이 중단된다.

로그의 A -> B, B -> A 의존 관계는 각 스레드가 이미 하나의 Lock을 보유한 채 상대 스레드가 가진 Lock을 기다리는 순환 대기를 직접 보여준다. 이 경우 해당 작업은 스스로 진행할 수 없으며, 단순히 PID가 존재한다는 사실만으로 프로그램이 정상 동작한다고 판단할 수 없다.

4. Workaround & Verification (조치 및 검증)

MULTI_THREAD_ENABLE을 true에서 false로 변경했다. 최종 실행에서는 THREAD Concurrency가 False [OK]로 표시되고 Healthy System Monitoring 시나리오가 선택되었다. Thread - A/B/C가 BLOCKED 없이 진행되었으며 이후 CPU cooldown과 메모리 cleanup도 정상 수행되어 프로세스가 지속 실행되었다.

내부 소스를 확인하지 않았으므로 false 설정이 Lock 자체를 제거했다고 단정하지 않는다. 관측 가능한 결론은 멀티스레드 Deadlock 시나리오가 더 이상 재현되지 않고 작업이 정상 진행되었다는 것이다.

```

agent-admin@kimseyoon:~$ export MULTI_THREAD_ENABLE=false
./agent-leak-app-arm64
>>> Starting Agent Boot Sequence...
[1/6] Checking User Account [OK]
... Running as service user 'agent-admin' (uid=1001)
[2/6] Verifying Environment Variables [OK]
... All required Envs correct
[3/6] Checking Required Files [OK]
... Verified 'secret.key' with correct key string.
[4/6] Checking Port Availability [OK]
... Port 15034 is available.
[5/6] Verifying Log Permission [OK]
... Log directory is writable: /var/log/agent-app
[6/6] Verifying Mission Environment [OK]
... MEMORY_LIMIT=512MB, CPU_MAX_OCCUPY=40%, MULTI_THREAD_ENABLE=False
-----
ALL Boot Checks Passed!
Agent READY
2026-09-19 14:35:05,482 [INFO] [SafetyGuard] Process priority lowered (nice=10).
2026-09-19 14:35:05,482 [INFO] Agent listening at port 15034

=====
[ Agent Initiate ] Resource Check
=====
[ MEMORY ] Limit: 512MB [ OK ]
[ CPU ] Limit: 40% [ OK ]
[ THREAD ] Concurrency: False [ OK ]
=====
>>> SYSTEM STATUS: STABLE, STARTING WORKLOAD MONITORING...
=====
2026-09-19 14:35:07,493 [INFO] >>> Scenario Selected: [Healthy System Monitoring]
>>> [SYSTEM] ALL CONFIGURATIONS OPTIMAL, RUNNING STABILITY TEST... <<<
2026-09-19 14:35:07,493 [INFO] [Scheduler] Task Scheduler Initialized.
2026-09-19 14:35:07,493 [INFO] [Scheduler] Registered Tasks: ['Thread-A', 'Thread-B', 'Thread-C']
2026-09-19 14:35:07,493 [INFO] [Scheduler] Starting task execution...
2026-09-19 14:35:07,493 [INFO] [Thread-A] Task Started. Calculating... (20%)
2026-09-19 14:35:07,546 [INFO] [Thread-A] Calculating... (40%)
2026-09-19 14:35:07,597 [INFO] [Thread-A] Preempted. Progress saved at (40%)
2026-09-19 14:35:07,648 [INFO] [Thread-B] Task Started. Calculating... (20%)
2026-09-19 14:35:07,699 [INFO] [Thread-B] Calculating... (40%)
2026-09-19 14:35:07,753 [INFO] [Thread-B] Preempted. Progress saved at (40%)
2026-09-19 14:35:07,804 [INFO] [Thread-C] Task Started. Calculating... (20%)
2026-09-19 14:35:07,855 [INFO] [Thread-C] Calculating... (40%)
2026-09-19 14:35:07,906 [INFO] [Thread-C] Preempted. Progress saved at (40%)
2026-09-19 14:35:07,958 [INFO] [Thread-A] Resumed. Calculating... (60%)
2026-09-19 14:35:08,013 [INFO] [Thread-A] Calculating... (80%)
2026-09-19 14:35:08,065 [INFO] [Thread-A] Preempted. Progress saved at (80%)
2026-09-19 14:35:08,118 [INFO] [Thread-B] Resumed. Calculating... (60%)
2026-09-19 14:35:08,169 [INFO] [Thread-B] Calculating... (80%)
2026-09-19 14:35:08,221 [INFO] [Thread-B] Preempted. Progress saved at (80%)
2026-09-19 14:35:08,275 [INFO] [Thread-C] Resumed. Calculating... (60%)
2026-09-19 14:35:08,328 [INFO] [Thread-C] Calculating... (80%)
2026-09-19 14:35:08,380 [INFO] [Thread-C] Preempted. Progress saved at (80%)
2026-09-19 14:35:08,431 [INFO] [Thread-A] Resumed. Calculating... (100%)
2026-09-19 14:35:08,483 [INFO] [Thread-B] Resumed. Calculating... (100%)

```

그림 10. MULTI_THREAD_ENABLE=false: 모든 설정 [OK], 작업이 BLOCKED 없이 정상 진행

최종 검증 요약

항목	Before	조치	After
OOM	256MB - > 275MB에서 종료	MEMORY_LIMIT=512MB	525MB 후 cleanup / 계속 실행
CPU	80% 조건 - > Watchdog SIGTERM	CPU_MAX_OCCUPY=40%	Peak 후 cooldown / 계속 실행
Deadlock	true - > WAITING/BLOCKED	MULTI_THREAD_ENABLE=false	Deadlock 시나리오 미재현 / 계속 실행

최종 정상 설정은 MEMORY_LIMIT=512MB, CPU_MAX_OCCUPY=40%, MULTI_THREAD_ENABLE=false이다. 이 조건에서 프로그램은 Healthy System Monitoring 상태로 진입했으며 CPU 부하는 cooldown으로 제어되고, 메모리 한계 도달 시 cleanup과 cache flush를 수행한 뒤 계속 실행되었다. Deadlock을 유발했던 상호 Lock 대기 역시 재현되지 않았다.

추가 고찰 - 실제 운영 환경 관점

1. 실제 운영 서버에서 monitor.sh를 어떻게 개선할 것인가?

현재 monitor - leak.sh는 실행한 시점의 CPU와 RSS를 기록하므로, 실제 운영 서버에서는 CPU와 RSS를 일정 주기로 자동 수집하도록 개선할 것이다. 장애는 예측하지 못한 시점에 발생할 수 있어 사람이 직접 스크립트를 실행하는 방식으로는 장애 직전의 변화를 놓칠 수 있기 때문이다. 또한 CPU나 메모리가 임계치를 초과하면 경고를 발생시키고, 장애 전후의 로그를 충분히 보존하도록 하겠다.

2. OOM, CPU Spike, Deadlock 중 실제 서버 환경에서 가장 치명적이라고 판단한 장애

세 장애 모두 서비스에 큰 영향을 줄 수 있지만, 본 실습에서는 OOM을 가장 치명적인 장애로 판단했다. Deadlock은 해당 작업의 진행을 멈추게 하고 CPU 과점유도 다른 작업의 응답성을 저하시킬 수 있다. 반면 지속적인 메모리 누수로 시스템의 가용 메모리가 고갈되면 해당 프로세스뿐 아니라 같은 서버에서 실행되는 다른 프로세스의 메모리 할당에도 영향을 줄 수 있기 때문이다. 근본적으로는 불필요한 동적 메모리 할당을 줄이고, 사용이 끝난 메모리를 적절히 해제하며, 캐시가 제한 없이 증가하지 않도록 관리해야 한다.

3. 동일 서버에서 OOM과 Deadlock이 동시에 발생한다면?

먼저 메모리 사용량과 시스템의 가용 메모리를 확인하여 메모리 고갈이 시스템 전체 장애로 확산되는 것을 우선 방지하겠다. Deadlock은 해당 작업을 멈추게 하지만, 진행 중인 메모리 고갈은 같은 서버의 다른 프로세스까지 영향을 줄 수 있기 때문이다. 시스템의 메모리 상태를 안정화한 뒤 PID 존재 여부, ps -L의 스레드 상태, WAITING/BLOCKED 로그를 순서대로 확인하여 Deadlock 원인을 분석하겠다.

4. 환경변수 조정이 아니라 소스 코드를 수정할 수 있다면?

OOM: 불필요한 동적 메모리 할당을 줄이고, 사용이 끝난 메모리를 적절히 해제하며 캐시 크기에 상한을 두어 메모리가 지속적으로 증가하지 않도록 개선한다.

CPU Spike: 불필요한 반복 연산이나 busy loop 등 CPU를 과도하게 점유하는 코드의 원인을 찾아 작업량을 줄이도록 개선한다.

Deadlock: 여러 Lock을 사용하는 경우 스레드들의 Lock 획득 순서를 통일하거나 Lock 범위를 줄여 Circular Wait가 발생하지 않도록 설계한다.

이번 실습의 환경변수 조정은 장애를 재현하거나 회피하기 위한 임시 조치이며, 실제 운영 환경에서는 장애의 원인이 되는 코드 자체를 수정하는 것이 근본적인 해결책이라고 생각한다.

5. 다시 이 미션을 처음부터 수행한다면?

먼저 정상 상태의 CPU와 메모리 값을 baseline으로 기록하고, 이후 환경변수를 한 번에 하나씩만 변경하여 다른 조건을 동일하게 유지한 상태에서 Before & After를 비교하겠다. 또한 top, ps 등의 OS 관측 결과도 가능한 경우 텍스트 로그로 함께 저장하여 장애 발생 순간마다 수동으로 화면을 캡처해야 하는 불편을 줄이겠다. 프로그램 내부 로그와 OS 관측 결과를 동일한 timestamp 기준으로 비교할 수 있도록 증거 수집 과정을 정리하면 원인과 결과를 더 명확하게 연결할 수 있을 것이다.